In [100…
```
# Libraries for analysis and modelling
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_scor
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeRegressor
```

In [101…
```
# Load the dataset and display the first rows to understand the structure
df = pd.read_csv("/Users/memnon/Downloads/global_housing_market_extended (1)
```

In [102…
```
# CHECK DATA
df.head()
df.shape
df.info()
df.describe()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 40 entries, 0 to 39
Data columns (total 11 columns):
 #   Column                 Non-Null Count  Dtype  
---  ------                 --------------  -----  
 0   Country                40 non-null     str    
 1   Year                   40 non-null     int64  
 2   House Price Index      40 non-null     float64
 3   Rent Index             40 non-null     float64
 4   Affordability Ratio    40 non-null     float64
 5   Mortgage Rate (%)      40 non-null     float64
 6   Inflation Rate (%)     40 non-null     float64
 7   GDP Growth (%)         40 non-null     float64
 8   Population Growth (%)  40 non-null     float64
 9   Urbanization Rate (%)  40 non-null     float64
 10  Construction Index     40 non-null     float64
dtypes: float64(9), int64(1), str(1)
memory usage: 3.7 KB
```

Out[102]:

	Year	House Price Index	Rent Index	Affordability Ratio	Mortgage Rate (%)	Inflation Rate (%)	GI Grow (%
count	40.000000	40.000000	40.000000	40.000000	40.000000	40.000000	40.0000
mean	2019.500000	132.682411	82.386097	7.312721	4.337135	3.843765	1.8467
std	2.908872	25.323249	22.388893	2.738655	1.480530	1.894218	2.3664
min	2015.000000	80.552212	50.354311	3.160865	1.621580	0.740550	-1.8298
25%	2017.000000	113.714969	62.389387	5.045168	3.171534	2.145478	-0.5548
50%	2019.500000	132.889365	77.814892	7.672790	4.515508	4.067000	1.9915
75%	2022.000000	152.312279	102.533826	9.402012	5.648986	5.326112	3.5702
max	2024.000000	179.005385	119.855388	11.729189	6.438612	6.783256	5.7566

In [103…
```
# Check missing values in each column
df.isnull().sum()

# Check duplicated rows in the dataset
df.duplicated().sum()

# Check countries in the dataset
df["Country"].unique()

# Count rows for each country
df["Country"].value_counts()
```

Out[103]:
```
Country
USA       10
UK        10
India     10
Brazil    10
Name: count, dtype: int64
```

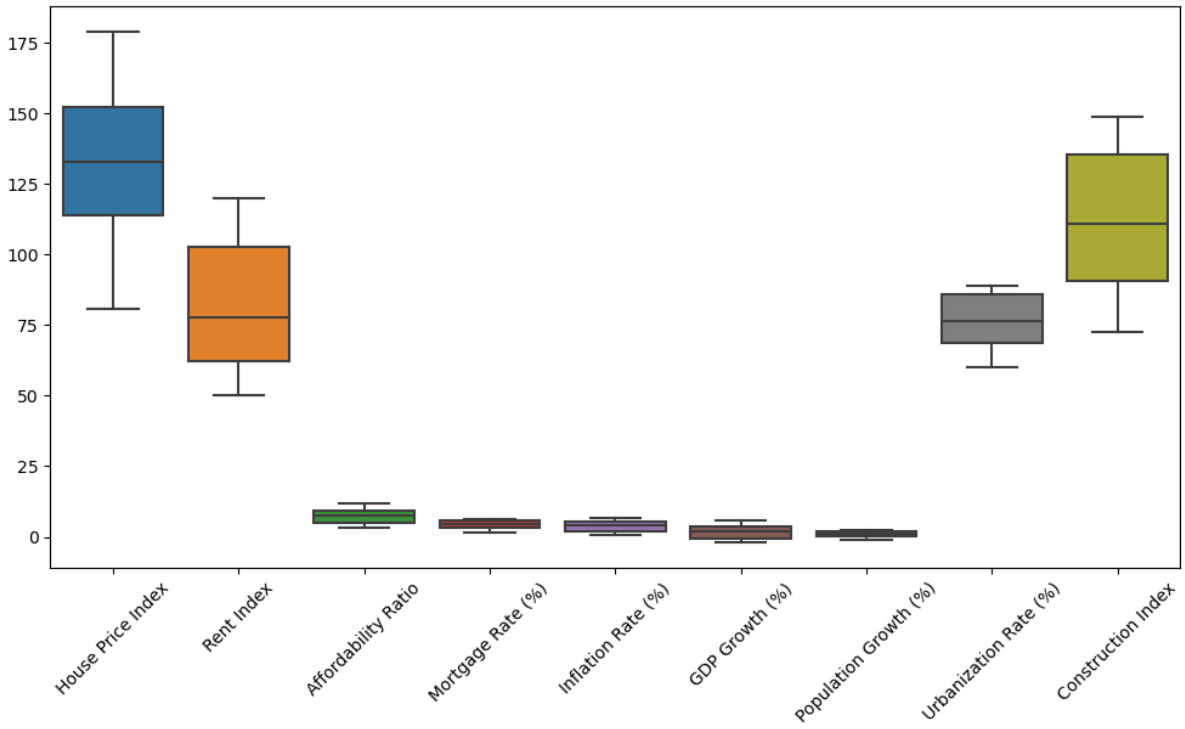
In [104…
```
# Create economy group

df["Economy Group"] = df["Country"].replace({
    "USA": "Developed",
    "UK": "Developed",
    "India": "Developing",
    "Brazil": "Developing"
})

# Check economy groups and year range
print(df["Economy Group"].value_counts())
print(df.groupby("Country")["Year"].agg(["min", "max", "count"]))

# Select numerical columns
numeric_columns = df.select_dtypes(include=["float64", "int64"]).columns
numeric_columns
```

```
Economy Group
Developed     20
Developing    20
Name: count, dtype: int64
          min   max  count
Country
Brazil   2015  2024     10
India    2015  2024     10
UK       2015  2024     10
USA      2015  2024     10
```

Out[104]:
```
Index(['Year', 'House Price Index', 'Rent Index', 'Affordability Ratio',
       'Mortgage Rate (%)', 'Inflation Rate (%)', 'GDP Growth (%)',
       'Population Growth (%)', 'Urbanization Rate (%)', 'Construction Ind
ex'],
      dtype='str')
```

In [105…

```
# Check outliers for housing market indicators

housing_columns = [
    "House Price Index",
    "Rent Index",
    "Affordability Ratio",
    "Mortgage Rate (%)",
    "Inflation Rate (%)",
    "GDP Growth (%)",
    "Population Growth (%)",
    "Urbanization Rate (%)",
    "Construction Index"
]

plt.figure(figsize=(12, 6))
sns.boxplot(data=df[housing_columns])
plt.xticks(rotation=45)
plt.show()
```



In [106…

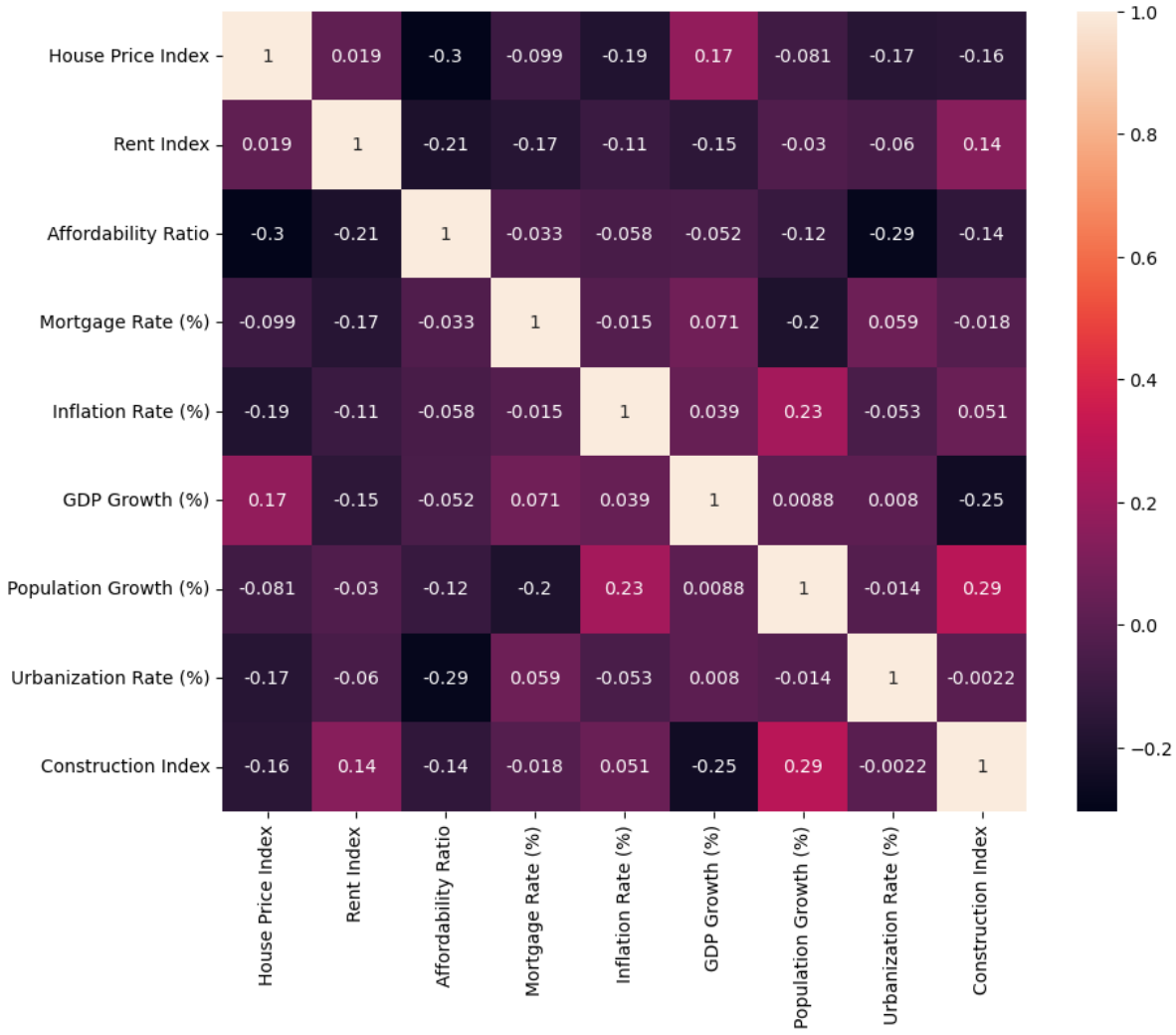
```
# Correlation between housing market variables

corr = df[housing_columns].corr()
corr
```

Out[106]:

	House Price Index	Rent Index	Affordability Ratio	Mortgage Rate (%)	Inflation Rate (%)	GDP Growth (%)	Populat Gro
House Price Index	1.000000	0.019058	-0.304630	-0.099382	-0.188851	0.173028	-0.080
Rent Index	0.019058	1.000000	-0.205180	-0.168719	-0.105640	-0.150267	-0.030
Affordability Ratio	-0.304630	-0.205180	1.000000	-0.033437	-0.057526	-0.052497	-0.116
Mortgage Rate (%)	-0.099382	-0.168719	-0.033437	1.000000	-0.014660	0.071448	-0.201
Inflation Rate (%)	-0.188851	-0.105640	-0.057526	-0.014660	1.000000	0.039440	0.227
GDP Growth (%)	0.173028	-0.150267	-0.052497	0.071448	0.039440	1.000000	0.008
Population Growth (%)	-0.080553	-0.030416	-0.116016	-0.201106	0.227582	0.008762	1.000
Urbanization Rate (%)	-0.170881	-0.059546	-0.286437	0.058733	-0.052931	0.008036	-0.014
Construction Index	-0.158244	0.143498	-0.139063	-0.017860	0.051238	-0.245887	0.289

In [107…

```
# Visualize correlation matrix
plt.figure(figsize=(10, 8))
sns.heatmap(corr, annot=True)
plt.show()
```

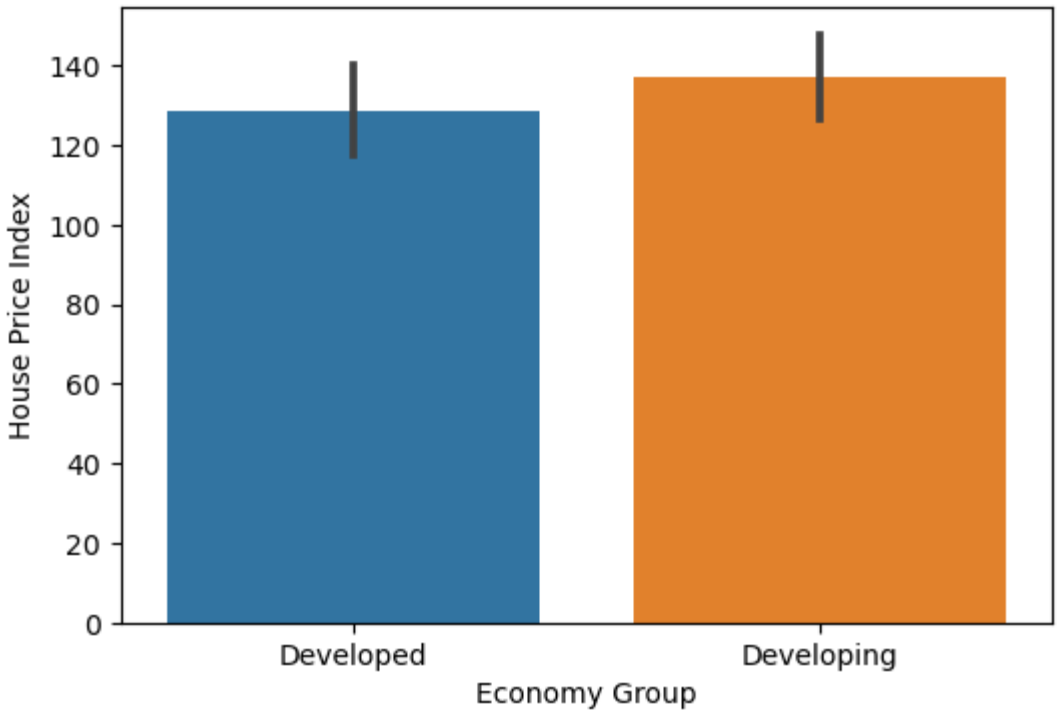


In [108…
```
# Compare developed and developing economies
group_summary = df.groupby("Economy Group")[housing_columns].mean()
group_summary
```

Out[108]:

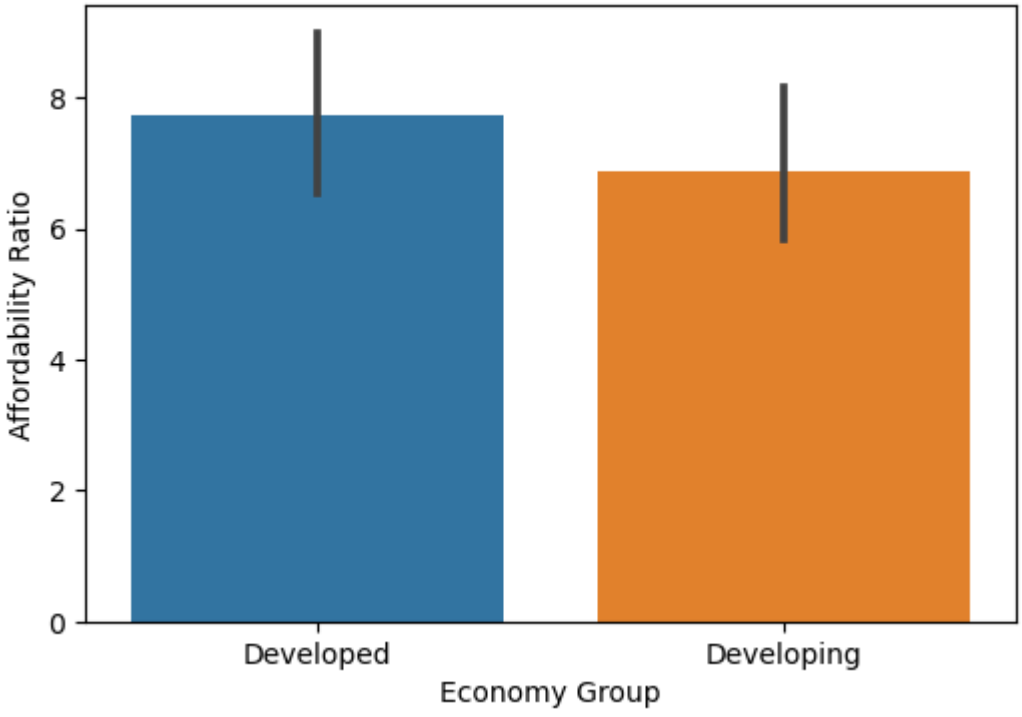
Economy Group	House Price Index	Rent Index	Affordability Ratio	Mortgage Rate (%)	Inflation Rate (%)	GDP Growth (%)	Population Growth (%)
Developed	128.444451	79.817984	7.738727	4.145119	3.556354	1.953581	0.835226
Developing	136.920371	84.954210	6.886715	4.529151	4.131176	1.739907	1.231674

In [109…
```
# Compare house price index by economy group
plt.figure(figsize=(6, 4))
sns.barplot(data=df, x="Economy Group", y="House Price Index")
plt.xlabel("Economy Group")
plt.ylabel("House Price Index")
plt.show()
```

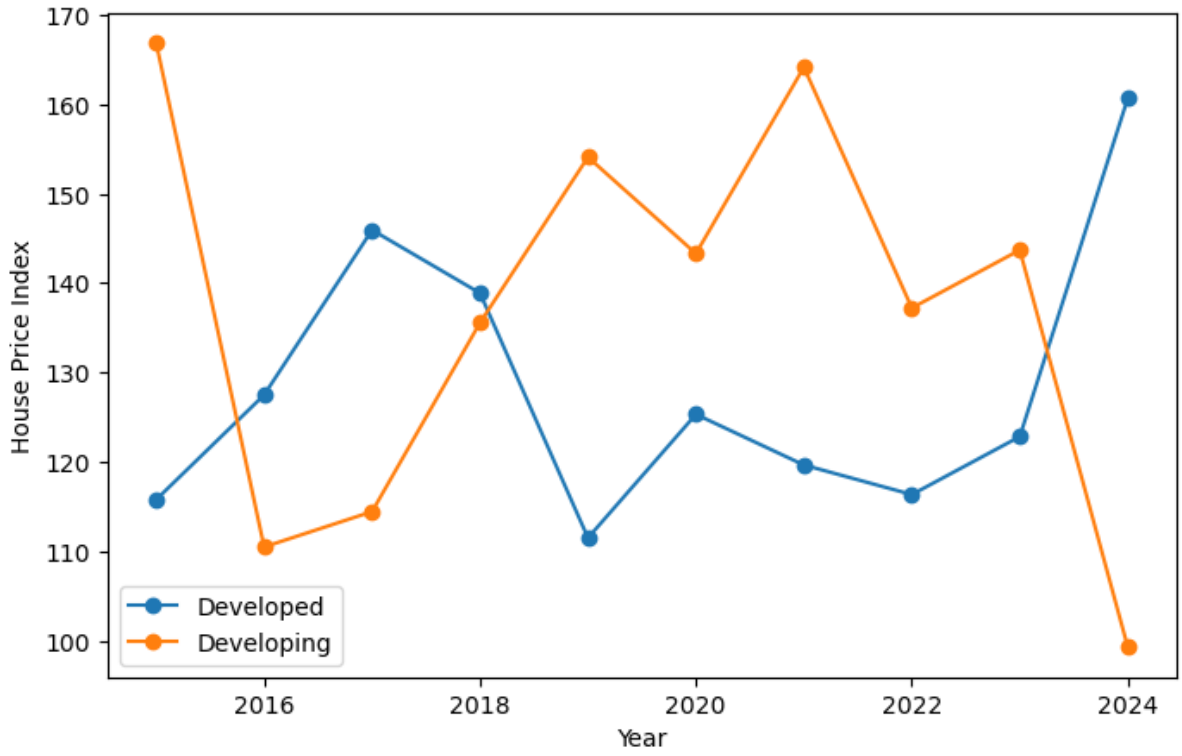


In [110…

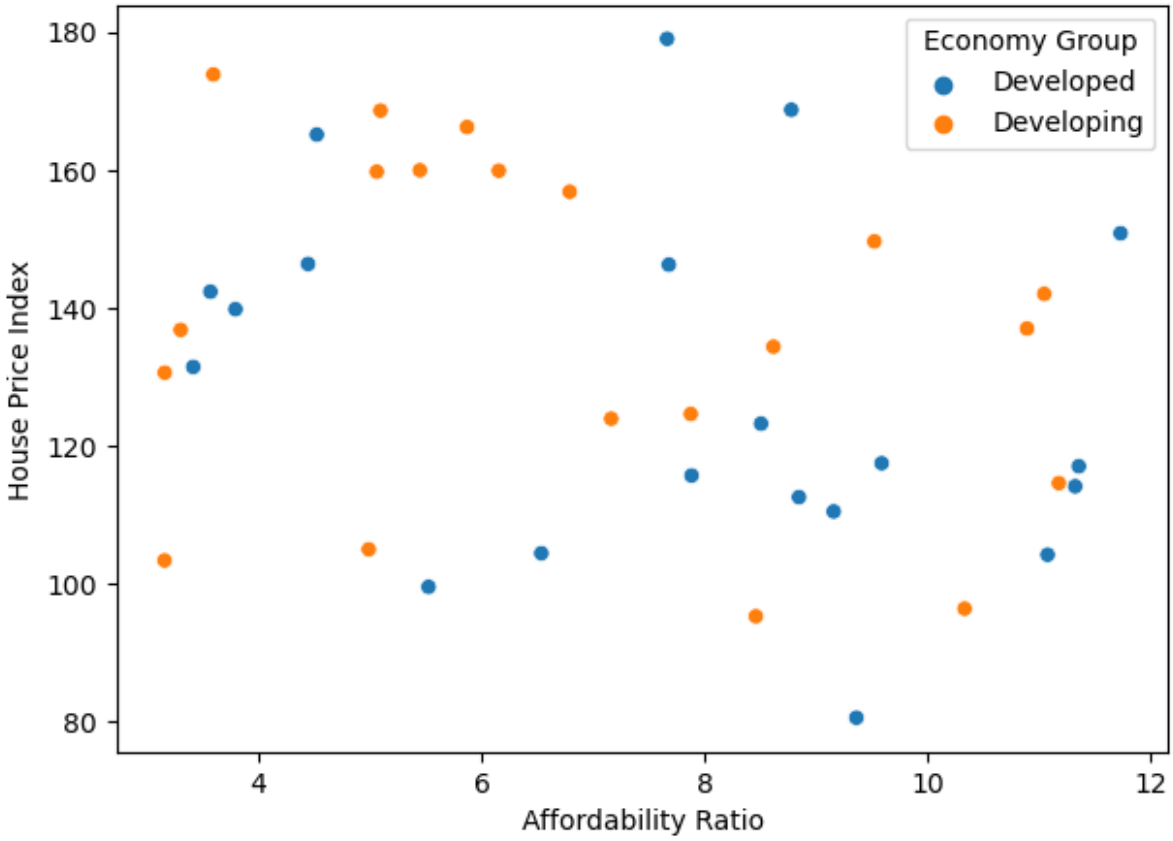
```
# Compare affordability ratio by economy group
plt.figure(figsize=(6, 4))
sns.barplot(data=df, x="Economy Group", y="Affordability Ratio")
plt.xlabel("Economy Group")
plt.ylabel("Affordability Ratio")
plt.show()
```



In [111…
```
# Compare house price trends over time
trend_data = df.groupby(["Year", "Economy Group"])["House Price Index"].mean

plt.figure(figsize=(8, 5))

for group in trend_data["Economy Group"].unique():
    data = trend_data[trend_data["Economy Group"] == group]
    plt.plot(data["Year"], data["House Price Index"], marker="o", label=grou

plt.xlabel("Year")
plt.ylabel("House Price Index")
plt.legend()
plt.show()
```



In [112…
```
# Relationship between house price index and affordability
plt.figure(figsize=(7, 5))
sns.scatterplot(
    data=df,
    x="Affordability Ratio",
    y="House Price Index",
    hue="Economy Group"
)

plt.xlabel("Affordability Ratio")
plt.ylabel("House Price Index")
plt.show()
```


Economy Group
Developed
Developing
House Price Index
Affordability Ratio
180
160
140
120
100
80
4
6
8
10
12

In [113…

```
# Define target and features
y = df["House Price Index"]

X = df.drop(columns=[
    "House Price Index",
    "Country",
    "Economy Group"
])

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train linear regression model
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

# Predict house price index
y_pred_linear = linear_model.predict(X_test)

# Evaluate linear regression model
print("MAE:", mean_absolute_error(y_test, y_pred_linear))
print("MSE:", mean_squared_error(y_test, y_pred_linear))
print("R2:", r2_score(y_test, y_pred_linear))

# Compare actual and predicted values
linear_results = pd.DataFrame({
    "Actual": y_test,
    "Predicted": y_pred_linear
})

linear_results
```

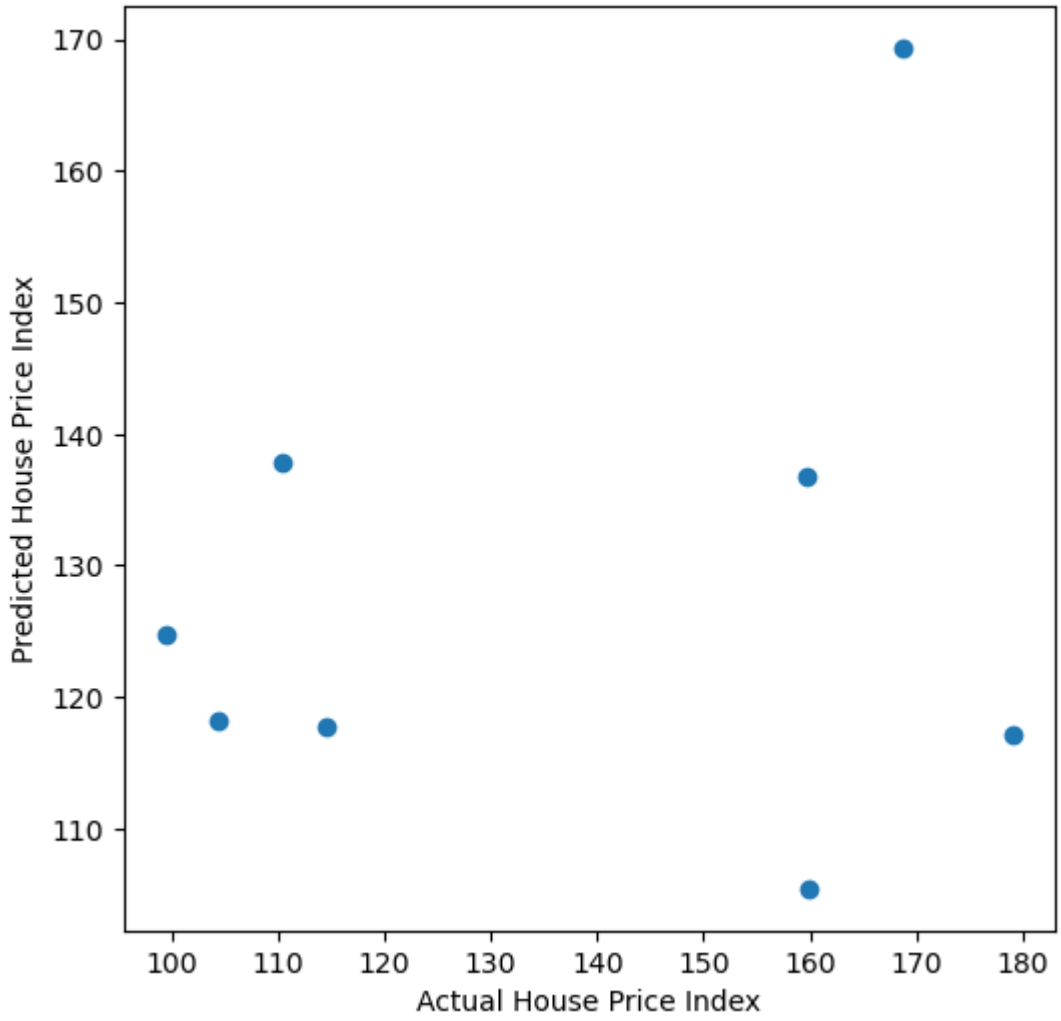
```
MAE: 26.194781731997402
MSE: 1113.8518091952615
R2: -0.1905520118246542
```

Out[113]:

	Actual	Predicted
19	179.005385	117.108764
16	99.524299	124.786404
15	104.398964	118.132170
26	159.729537	136.711129
4	110.461377	137.856902
12	168.708642	169.387046
37	159.878324	105.453041
27	114.569599	117.718302

In [114…

```
# Visualize actual vs predicted values
plt.figure(figsize=(6, 6))
plt.scatter(y_test, y_pred_linear)
plt.xlabel("Actual House Price Index")
plt.ylabel("Predicted House Price Index")
plt.show()
```



In [115…

```
# Train decision tree model
tree_model = DecisionTreeRegressor(random_state=42)
tree_model.fit(X_train, y_train)


# Predict with decision tree model
y_pred_tree = tree_model.predict(X_test)

# Evaluate decision tree model
print("MAE:", mean_absolute_error(y_test, y_pred_tree))
print("MSE:", mean_squared_error(y_test, y_pred_tree))
print("R2:", r2_score(y_test, y_pred_tree))


# Compare model performance
model_comparison = pd.DataFrame({
    "Model": ["Linear Regression", "Decision Tree"],
    "MAE": [
        mean_absolute_error(y_test, y_pred_linear),
        mean_absolute_error(y_test, y_pred_tree)
    ],
    "MSE": [
        mean_squared_error(y_test, y_pred_linear),
        mean_squared_error(y_test, y_pred_tree)
    ],
    "R2": [
        r2_score(y_test, y_pred_linear),
        r2_score(y_test, y_pred_tree)
    ]
})

model_comparison
```

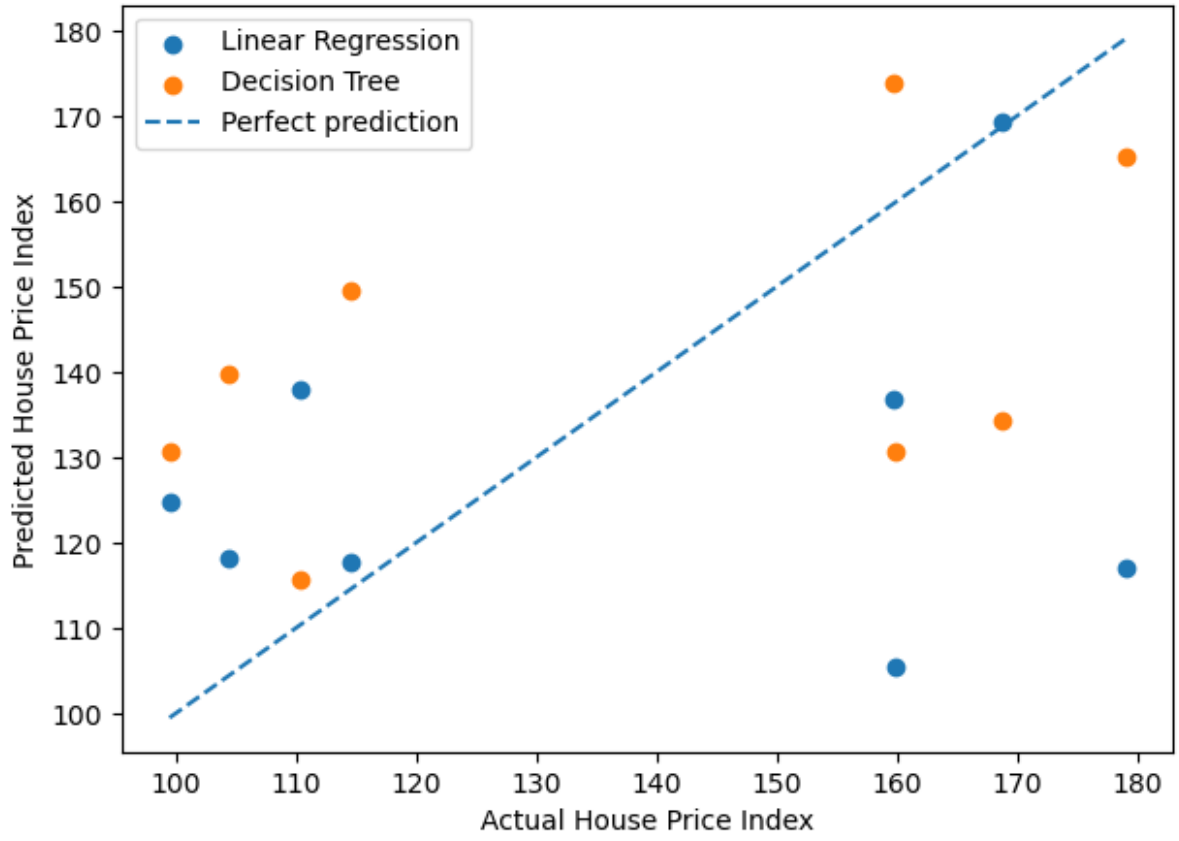
```
MAE: 24.795984452499997
MSE: 737.9766676925958
R2: 0.2112060157842297
```

Out[115]:

	Model	MAE	MSE	R2
0	Linear Regression	26.194782	1113.851809	-0.190552
1	Decision Tree	24.795984	737.976668	0.211206

In [116…

```
# Compare actual and predicted values for both models
plt.figure(figsize=(7, 5))

plt.scatter(y_test, y_pred_linear, label="Linear Regression")
plt.scatter(y_test, y_pred_tree, label="Decision Tree")

plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    linestyle="--",
    label="Perfect prediction"
)

plt.xlabel("Actual House Price Index")
plt.ylabel("Predicted House Price Index")
plt.legend()
plt.show()
```



In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: